UNIFAAT 2026.2

# 09 - Os 3 Pilares da Orientação a Objetos

**Prof. Luan Lourenço**

# O `Item` da Aula 08 aceita qualquer coisa

a classe aberta (Aula 08)

```
trabalhos.nome = 1   # o nome virou um número
trabalhos.filhos = "qualquer coisa"   # a árvore se perdeu
trabalhos.altura = 1.70   # propriedade que ninguém declarou

nenhuma dessas linhas dá erro
```

---

`objeto.propriedade = valor` de fora da classe é feio e perigoso: a classe perde o controle dos **próprios dados**. Hoje: como fechar essa porta.

# Quem pode ver o quê: modificadores de acesso

| nível | quem acessa | Java | Python |
|---|---|---|---|
| **público** | qualquer um | `public` | `self.nome` |
| **protegido** | a classe e as filhas | `protected` | `self._nome` (convenção) |
| **privado** | só a própria classe | `private` | `self.__nome` (renomeado) |

src/09/visibilidade.py

```
class ItemConvencao:
    def __init__(self, nome):
        self.nome = nome       # público
        self._pai = None       # protegido: só pede
        self.__filhos = []     # privado: o Python renomeia

item.__filhos   # AttributeError
item._ItemConvencao__filhos   # o nome renomeado: não é cofre
```

---

Em Java o **compilador proíbe**; em Python a **convenção pede**. Vale também para métodos: `_receber()` e `__validar_nome()`.

# Getter e setter: a porta controlada

src/09/encapsulamento.py

```python
@property
def nome(self):                # getter
    return self.__nome

@nome.setter
def nome(self, novo_nome):   # setter
    self.__validar_nome(novo_nome)
    self.__nome = novo_nome

@property
def pai(self):                 # só getter
    return self.__pai
```

| combinação | permite |
|---|---|
| getter + setter com validação | ler e escrever, com regra |
| só getter | só ler (`item.pai = x` dá erro) |
| **nem setter: comportamento** | `mover_para(destino)` |

---

Quem usa continua escrevendo `item.nome`. O encapsulamento mais forte nem libera setter: **expõe o que o objeto faz, não o dado cru**.

# Nenhum atributo novo: `__slots__`

```
# em Python o atributo nasce na atribuição
p.altura = 1.70 cria sem reclamar
os atributos moram no __dict__
__slots__ lista os únicos permitidos
e elimina o __dict__
```

src/09/encapsulamento.py

```python
class Item:
    __slots__ = ("__nome", "__eh_pasta",
                 "__pai", "__filhos")

    @property
    def filhos(self):
        return tuple(self.__filhos)  # cópia
```

- → `trabalhos.altura = 1.70` e `trabalhos.pai = None` agora levantam `AttributeError`
- → Devolver a lista de dentro **vaza** a cerca: `.filhos.append(x)` burlaria as regras. A tupla é uma cópia imutável
- → Nada é cofre: `trabalhos._Item__nome` ainda chega. A proteção é **por contrato**

# Antes dos pilares: o que é abstrair?

Do latim *abstrahere*, "arrastar para fora, separar": **recortar o essencial e ignorar o detalhe**. Não é inventar, é recortar.

```
# o que eu uso (interface)
mapa do metrô: estações e ligações
dirigir: volante, pedais, câmbio
Explorador: pastas e arquivos
```

```
# o que ignoro (implementação)
as ruas, as curvas, as distâncias
a combustão, a injeção
setores de disco, blocos, ponteiros
```

---

O ganho é sempre o mesmo: **usar sem precisar entender tudo**. Guarde a frase: ela volta em cada pilar.

pilar 1

# Encapsulamento: dados privados, porta pública

- Agrupar dados e métodos e **controlar o acesso**: `__`, `property`, `__slots__`
- Caixa eletrônico: você usa `sacar()`, não mexe no saldo
- A classe pode mudar por dentro sem quebrar quem a usa

---

**Abstração aqui:** separa a **interface** (`mover_para()`) da **implementação** (`__filhos`).

▶ Diagrama vivo: `aula-09-pilares/modelos/encapsulamento/index.html`

pilar 2

# Herança: o geral e o específico

- → Relação **"é um"**: `Pasta` é um `Item`, `Arquivo` é um `Item`; cada filha lista só o que acrescenta
- → Acoplamento forte: só com "é um" de verdade. "Tem um" é **composição** (a pasta *tem* filhos)
- → `_receber()` protegido: o que a filha enxerga (e sobrescreve) da mãe

---

**Abstração aqui:** separa o **geral** (`Item`) do **específico** (`Pasta`, `Arquivo`).

▶ Diagrama vivo: `aula-09-pilares/modelos/heranca/index.html`

pilar 3

# Polimorfismo: a mesma chamada, formas diferentes

- → `filho.tamanho()` serve para pasta e arquivo, **sem** `if eh_pasta`
- → **Override**: a filha redefine o método e o Python escolhe em execução. **Python não tem overload**
- → Sem herança também vale: **duck typing** (quem tem `tamanho()` serve)

---

**Abstração aqui:** separa **o que se pede** (`tamanho()`) de **como é feito** (cada classe calcula).

▶ Diagrama vivo: `aula-09-pilares/modelos/polimorfismo/index.html`

# Os 3 pilares num relance: a abstração dentro de cada um

| pilar | a abstração embutida | no código | diagrama vivo |
|---|---|---|---|
| **encapsulamento** | separa a **interface** da **implementação** | `mover_para()` por fora, `__filhos` por dentro | ▶ `modelos/encapsulamento` |
| **herança** | separa o **geral** do **específico** | `Item` × `Pasta` e `Arquivo` | ▶ `modelos/heranca` |
| **polimorfismo** | separa **o que se pede** de **como é feito** | `tamanho()` pedido, cada classe calcula | ▶ `modelos/polimorfismo` |

---

Nos três, o movimento é o mesmo: **ignorar o detalhe para trabalhar com o essencial**. A abstração não está ao lado dos pilares: está **misturada dentro** de cada um.

# Interface: a tomada da parede

```
# a tomada
tem um formato (os furos, a voltagem)
não sabe o que vai ser ligado:
abajur, carregador, liquidificador
```

```
# o aparelho
segue o mesmo formato de plugue
não sabe de onde vem a energia:
hidrelétrica, painel solar, gerador
```

Os dois se entendem pelo **contrato**: o formato do plugue.

---

Interface é só a lista de métodos, **sem estado e sem implementação**: diz *o que*, nunca *como*.

# O contrato Medivel na árvore

src/09/pilares.py

```
class Medivel(ABC):          # o plugue
    @abstractmethod
    def tamanho(self): pass

class Item(Medivel): ...
class Lixeira(Medivel): ...  # fora da árvore

def espaco_total(coisas):      # a tomada
    total = 0
    for coisa in coisas:
        total = total + coisa.tamanho()
    return total

espaco_total([notas, trabalhos, lixeira])
5960
```

```
# Java: a mesma ideia
interface Medivel {
    long tamanho();
}
class Lixeira implements Medivel

# Python: também Protocol (duck typing)
Medivel() → TypeError: classe abstrata
```

---

Nos três pilares, a abstração aparece misturada. **Na interface, ela aparece nua.**

# São 3 ou 4 pilares?

- A **abstração é um pilar** (tem manifestação própria na interface) ou o **chão** onde os outros três se apoiam?
- O **encapsulamento** é recurso da linguagem ou **ideia de projeto**? O Python tem a ideia sem o recurso e dá para ir apertando a cerca
- **3 ou 4** são ambos válidos na literatura: nomear a abstração ajuda a nomear o raciocínio

---

A conclusão é **de cada um**. Python é OO no modelo e permissivo na disciplina: *não é OO, ou só não me obriga a ser?*

tarefa

# TF 09: o Atalho na árvore

Num repositório novo `tf-09`, com venv e `.gitignore`, partir do `pilares.py` e criar a classe `Atalho(Item)`:

- → **Encapsulamento:** `__destino` privado, em `__slots__`, só leitura; trocar só por `redirecionar()`
- → **Polimorfismo:** `tamanho()` e `mostrar()` próprios; o `espaco_total()` segue igual
- → **Herança e interface:** nada de `nome`, `pai` ou `caminho` copiado; vira `Medivel` de graça
- → 3 tentativas que **falham** de propósito, `README` sobre interface x implementação e 4+ commits
- → Sem `else`, sem `isinstance`, `.venv` fora do repositório

Entrega: link do repositório público no GitHub, pelo formulário abaixo — `docs.google.com/forms/d/e/1FAIpQLSdBiL-WCzYTcTK_BzVNVoMQqNtPtzK3o9QldB5wSOBt0HJC8w/viewform`

---

Prazo: **28/10, até 23:59**.